

Implementation of Combinatorial n-fold Algorithms

Bachelor Thesis Defense

Presented by:

Lennart Klose

Department of Computer Science

Kiel University

stu222821@mail.uni-kiel.de

Supervisor: Prof. Dr. Klaus Jansen

Co-Supervisor: Lis Pirotton

21.09.2026

Introduction

- Implementing Algorithms for combinatorial n-fold ILPs
- General Problem has form $\min\{c^T x \mid \mathcal{A}x = b, x \in \mathbb{Z}_{\geq 0}^n\}$
- Used to model many discrete problems such as:
 - Scheduling
 - Packing
 - Allocation
- General ILPs are computationally difficult
- Special structure can be exploited to design efficient algorithms

Motivation and Goal

- Four recent algorithms for combinatorial n-fold ILPs
- Different theoretical approaches and runtime bounds
- Practical performance can differ from theoretical guarantees
 - Memory requirements are rarely considered
- **Goal:** Implement and experimentally compare all four algorithms
- **Focus:** Runtime, scalability, and success rates

Combinatorial n-fold ILP

- Let $r, t, n \in \mathbb{Z}_{\geq 0}$. A combinatorial n-fold ILP is an integer linear program with right-hand side vector $b \in \mathbb{Z}_{\geq 0}^{r+n}$ and a constraint matrix $\mathcal{A} \in \mathbb{Z}_{\geq 0}^{(r+n) \times (n \cdot t)}$ of the form:

$$\mathcal{A} = \begin{pmatrix} A_1 & A_2 & \cdots & A_n \\ \mathbf{1}_t & 0 & \cdots & 0 \\ 0 & \mathbf{1}_t & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \mathbf{1}_t \end{pmatrix}.$$

- Let m denote the number of rows for convenience

Preliminaries

- For a matrix M or a vector v , denote the maximum entry with Δ_M and Δ_v
- Δ is used to denote $\Delta_{\mathcal{A}}$
- v_k refers to the k^{th} entry of a vector v
- Split b : first r entries are $b^\uparrow$ or upper right-hand side, last n entries are $b^\downarrow$, or lower right-hand side
- Several algorithms use the Steinitz Lemma:

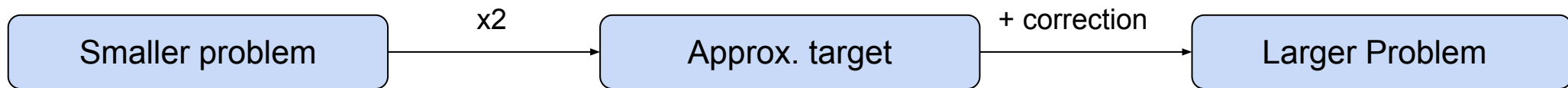
Let $|\cdot|$ be a norm on $\mathbb{R}^m$, and let $v^{(1)}, \dots, v^{(t)} \in \mathbb{R}^m$ satisfy $|v^{(i)}| \leq \Delta$ for all $i \in [t]$ and $\sum_{i=1}^t v^{(i)} = 0$. Then, there exists a permutation $\pi \in S_t$ such that, for all $j \in \{1, \dots, t\}$,

$$\left| \sum_{i=1}^j v^{(\pi(i))} \right| \leq m\Delta.$$

Overview of the Algorithms

- Jansen et al. (JKPT25) — Divide-and-conquer approach
Running time of $(nr\Delta)^{\mathcal{O}(r)} \log \|b\|_\infty$
- Jansen & Rohwedder (JR) — Discrepancy and (min,+)-convolution
Running time of $\mathcal{O}((\sqrt{m}\Delta)^{(1+o(1))m}) + \mathcal{O}(nm)$
- Gupta et al. (GJSZ25) — Steinitz Lemma-based approach
Running time of $\mathcal{O}(nt \cdot (n\Delta(n+1+4r))^r \cdot q)$
- Rohwedder (Rohwedder25) — Scheduling-based approach
Running time of $p_{\max}^{\mathcal{O}(d)}$

JKPT25 Algorithm – Overview

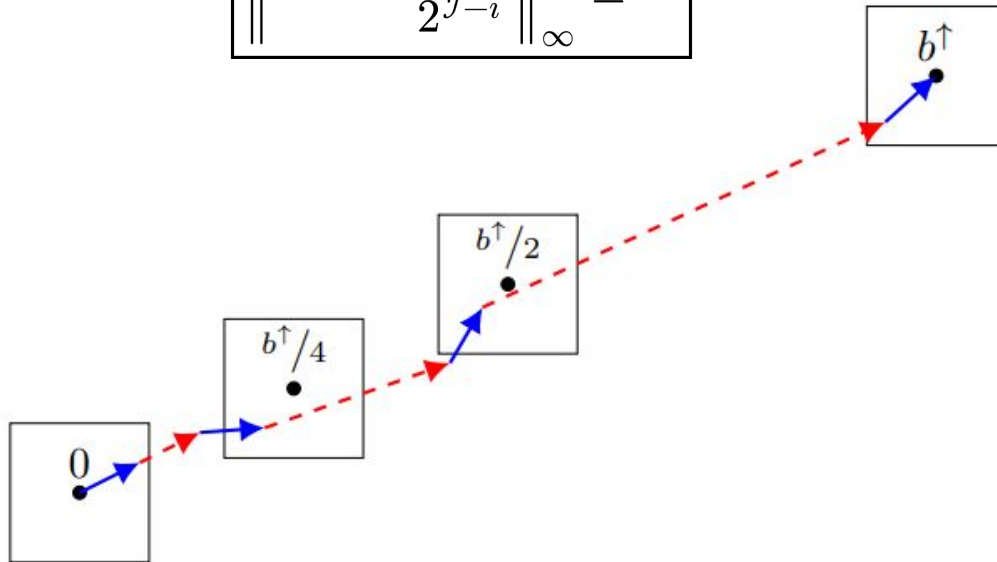


$$\text{Target at iteration } i : \frac{b^\uparrow}{2^{\mathcal{I}-i}}$$

- Recursively construct a solution to $\mathcal{A}x = b$
- Double a solution from the previous iteration
- Add a small correction to compensate for the difference
- Repeat for $\mathcal{I}$ iterations

JKPT25 Algorithm – Bounded Intermediate States

$$\left\| b^{\uparrow(i)} - \frac{b^{\uparrow}}{2^{j-i}} \right\|_{\infty} \leq D$$



Source. Jansen et. al (2025), Fig. 1

Feasible if solution x exists to $\mathcal{A}x = (b^{\uparrow}, b^{\downarrow})^T$

Solve two subproblems:

- Small Problem $\|\tilde{b}^{\downarrow}\|_{\infty} \leq K$ $b^{\downarrow(i)} = \hat{b}^{\downarrow(i)} + \tilde{b}^{\downarrow(i)}$
- Even Problem $\hat{b}^{\downarrow}$ is even

Construct three sets:

- $\hat{N}^{(i)}$: doubled solutions from the previous iteration
- $\tilde{N}^{(i)}$: small corrections
- $N^{(i)}$: combinations within the D-box

JKPT25 Algorithm – Implementation

- Calculate $\tilde{b}^\downarrow$ for each iteration
- Use a dynamic program to calculate the upper right-hand side

$$BT(\nu, k) \leftarrow \begin{cases} \text{true,} & \text{if } A_k \tilde{x}_k = \nu \text{ with } \|\tilde{x}_k\|_1 = \tilde{b}_k^{\downarrow(i)}, \tilde{x}_k \in \mathbb{Z}_{\geq 0}^{t_i} \text{ is feasible,} \\ \text{false,} & \text{otherwise.} \end{cases}$$

$$DT(\nu, k) \leftarrow \bigvee_{\substack{\nu' + \nu'' = \nu \\ \nu', \nu'' \text{ integral vectors}}} DT(\nu', k-1) \wedge BT(\nu'', k)$$

- Encode vectors as as number of base K+1 or D+1 to use BitSets
- Filter early if target cannot be reached

JR Algorithm – Overview

- View vectors as points in the grid $\mathbb{Z}_{\geq 0}^r$
- Adding the vectors one at a time creates a walk from 0 to b
- The Steinitz Lemma gives the intuition that the vectors can be reordered so that the walk stays close to the path from 0 to b
- The distance to this midpoint is related to the hereditary discrepancy of $\mathcal{A}$, which gives us a bound H
- Guess the vector b' at each iteration $i \in [\mathcal{I}]$ and combine them with vectors of the previous iteration

JR Algorithm – Discrepancy and Convolution

- **Discrepancy:** Measures how evenly the columns of a matrix can be partitioned into two groups, based on the maximum imbalance across rows

$$\text{disc}(A) = \min_{z \in \{0,1\}^n} \left\| A \left(z - \left(\frac{1}{2}, \dots, \frac{1}{2} \right)^T \right) \right\|_{\infty}$$

- **Hereditary discrepancy:** Takes the maximum discrepancy over all subsets of columns of the matrix.

$$\text{herdisc}(A) = \max_{I \subseteq \{1, \dots, n\}} \text{disc}(A_I)$$

- **Convolution:** Combine two feasible vectors from iteration $i - 1$ whose sum gives b'

$$b' \text{ feasible at iteration } i \implies \exists b'' \text{ such that } b'' \text{ and } b' - b'' \text{ are feasible at level } i - 1$$

JR Algorithm – Implementation

- Calculate herDisc instead of using upper bound
 - **Computationally expensive! (exponential runtime)**
 - But: much better pruning of invalid states
- Use BitSets again, using $\max\{\Delta_A, \Delta_b\} + 1$ the base
- Pairwise convolution of candidates
 - If larger than b in any component b_k , we can discard the vector
 - No need to perform all iterations, as vectors stay in subsequent iterations

GJSZ25 Algorithm – Overview

- A solution x^* specifies how many columns are chosen from each block A_i
- Let q be the total number of chosen columns
 - The chosen columns sum of the components in the lower right-hand side $b^\downarrow$
- We can represent the chosen columns by a matrix M that records which block each chosen vector comes from
- The order of the chosen columns can be changed, affecting the intermediate partial sums
- Permutation M_σ exists where the the imbalance is bound by $-n \leq \text{imb}_{M_\sigma}(e, j) \leq 1$

$$\begin{pmatrix} 2 & 1 & 4 & 8 \\ 3 & 2 & 2 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} 4 \\ 2 \\ 1 \\ 5 \end{pmatrix}$$

GJSZ25 Algorithm – Overview (cont.)

- **Imbalance** measures the difference between the actual and proportionally expected number of occurrences of e in the first j columns of M : $\text{imb}_M(e, j) := \text{occ}_M(e, j) - (j/q) \cdot m_e$
- The Steinitz Lemma gives an ordering of the columns such that: $-n \leq \text{imb}_M \sigma(e, j) \leq 1$
- Thus, the columns can be ordered so that their partial sums remain close to their expected values
- For the reordered columns, every partial sum for each $j \in [q]$ and $k \in [r]$ satisfies:

$$(j/q)b_k^\uparrow - n\Delta(n + 2r) \leq v_k \leq (j/q)b_k^\uparrow + n\Delta(1 + 2r)$$

- This allows the state space to be limited
- Construct a weighted, layered graph G with vertices $h_{j,v}$
 - j represents the amount of columns chosen so far
 - v represents the partial-sum vector

GJSZ25 Algorithm – Implementation

- Gupta et. al provide two algorithms in their own paper to construct the solution
 - The first one calculates M_σ , where $M_\sigma[j] = i$ if the j^{th} column we add is from block i
 - The second one constructs the graph G
- Add edge (h_{j-1,v_1}, h_{j,v_2}) if $v_2 - v_1$ corresponds to a column of a block A_i
 - **Improvement:** Sum up the columns of A_i with v_2 to calculate v_1 !
- Only keep the resulting state if v_2 remains within the Steinitz bounds
- Finally, search the DAG for a path from the start to the target; for feasibility, we only check whether such a path exists.

Rohwedder25 Algorithm - Overview

- Originally about makespan minimization on uniform machines
- Let P be a partition of the variables into sets corresponding to the blocks of $\mathcal{A}$
- Start with $x = 0$ and perform $b_S^\downarrow$ increments, increasing one variable by 1 at each step
- For each set $S \in P$, construct the pairs

$$\left(\frac{j}{b_i^\downarrow}, S \right) \quad j = 1, \dots, b_i^\downarrow$$

- Sort all pairs by their first component to obtain an ordering $(d_1, S_1), \dots, (d_{b_S^\downarrow}, S_{b_S^\downarrow})$.
- This orders the increments in a balanced way, keeping the intermediate right-hand sides close to their proportional target

Rohwedder25 Algorithm - Overview (cont.)

- Construct a layered graph G
 - $b_S^\downarrow + 1$ sets of vertices $V_1, \dots, V_{b_S^\downarrow + 1}$
 - Vertex V_k contains all possible $b' \in \mathbb{Z}_{\geq 0}^r$ fulfilling $\|b' - d_k \cdot b\|_\infty \leq 4rn\Delta$
 - Edges are added if for $b' \in V_k$ and $b'' \in V_{k+1}$, $b'' - b'$ corresponds to a column of $\mathcal{A}$
 - Feasible if a path from 0 to b exists

Rohwedder25 Algorithm - Implementation

- Construct breakpoints: Generate and sort all $(j/b_i^\downarrow, S)$ to obtain the sequence $(d_1, S_1), \dots, (d_{b_S^\downarrow}, S_{b_S^\downarrow})$
- Build the graph layer by layer: Start with only the zero vector in the first layer
- Directly generate next states: Instead of checking every pair of vertices, for each current state b' and column $\mathcal{A}_{\{.,i\}}$ from the current block, compute $b'' = b' + \mathcal{A}_{\{.,i\}}$
- Pruning: Keep candidate right-hand sides only if they are within the bound and do not exceed b in any component
- Since every state is generated from the initial zero state, check whether b appears in the final layer

Input Generation

- Generate instances with different values of $\Delta_A, \Delta_b, n, t, r$.
- Use predefined ranges and step sizes to systematically vary parameters
- Same values used for n and t to limit the total number of instances
 - They both scale the column amount
- Motivation: identify the parameters that have the largest impact on runtime.

Parameter	Ranges
Δ_A	(1, 10, 1), (20, 100, 20), (500, 1000, 500)
Δ_b	(1, 10, 1), (20, 100, 20), (500, 1000, 500)
n, t	(2, 10, 2)
r	(1, 10, 1)

Batch Processing

- Early tests showed particularly strong growth with r , so instances were grouped in separate batches
- Experiments were executed on the CAU university cluster using Slurm, allowing parallel execution
- For each input, all four algorithms were run on the same instances

Runtime was measured by both:

- An internal program timer
- Slurm execution logs
- Results and logs were collected for subsequent comparison
- Timeouts: empty output after the input parameters
 - Five hour time limit
- Memory failures: indicated by a Java heap error

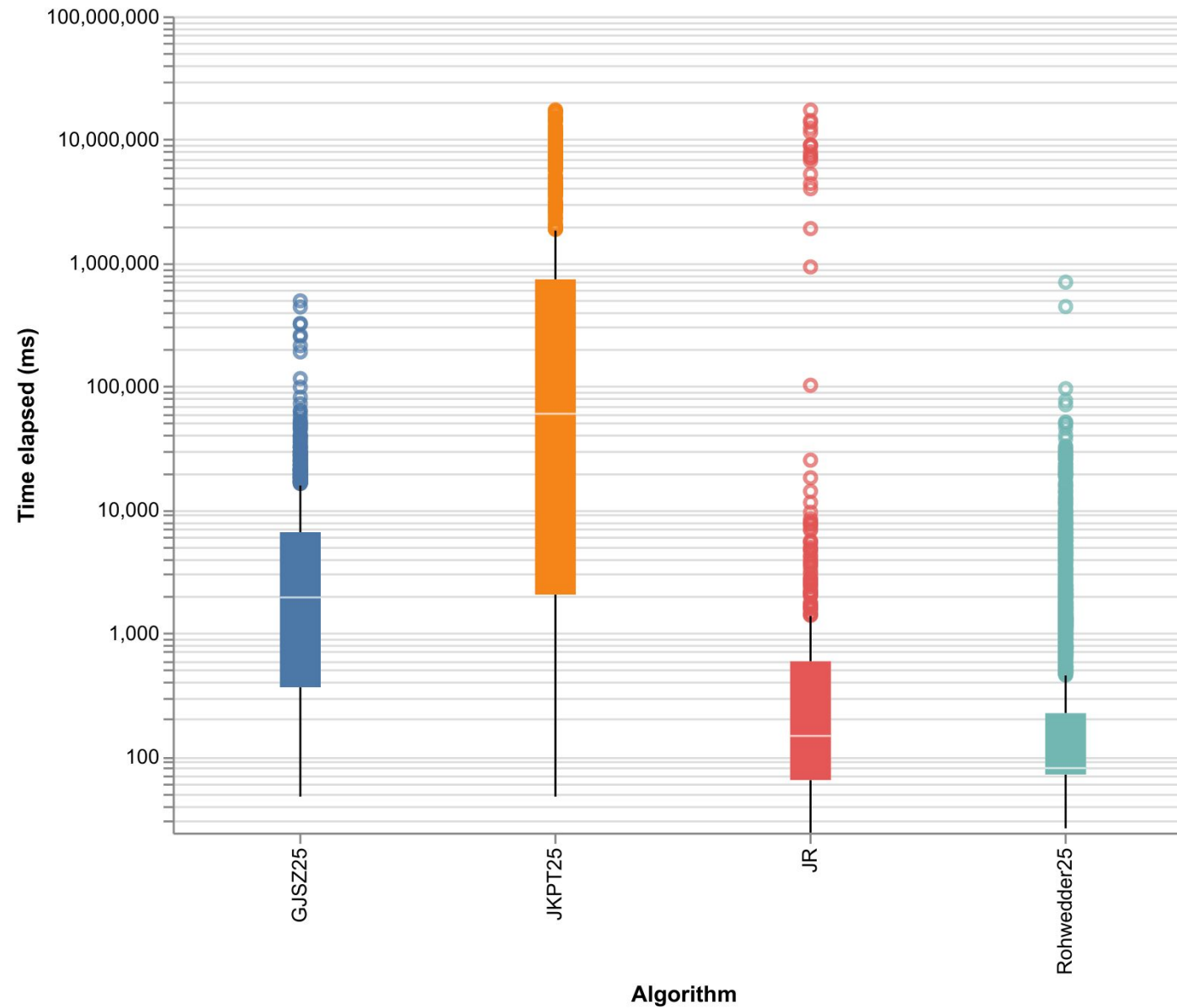
Overall performance comparison

- Benchmark results for **successful executions(!)**, while also stating their success rate:

Algorithm	Median runtime (ms)	Mean runtime (ms)	Success rate (%)
JKPT25	59716.00	1217371.52	7.79
JR	146.00	291548.25	4.98
GJSZ25	1937.50	8904.28	8.49
Rohwedder25	80.00	1489.38	27.59

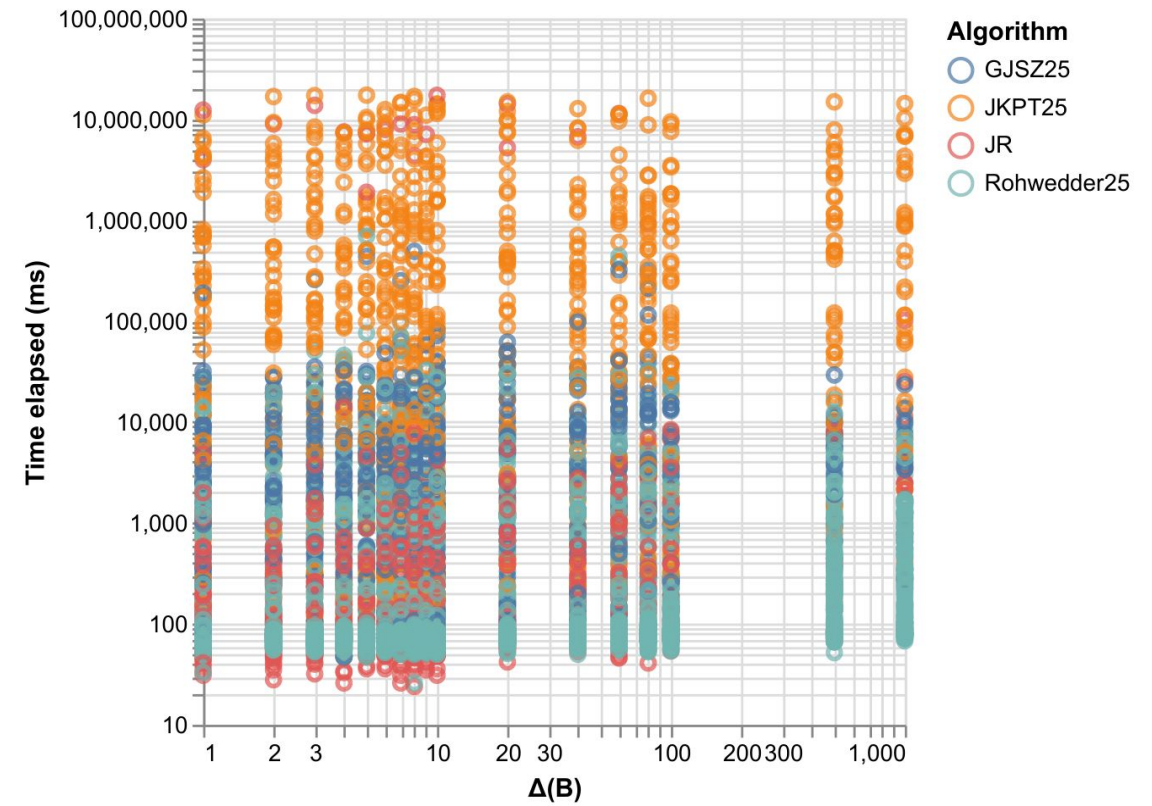
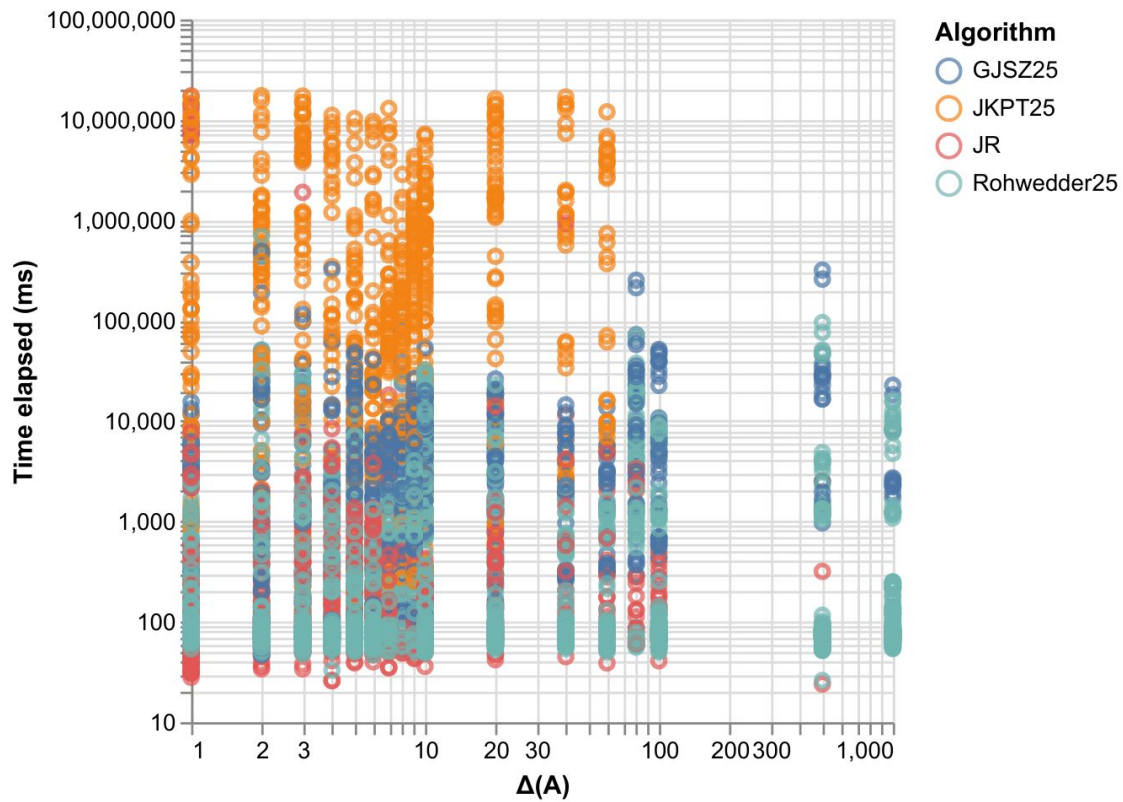
- Large runtime differences between the four algorithms
- Rohwedder25: lowest median and mean runtime, and highest success rate
- JR: low median runtime, but the lowest success rate limits its overall performance

Distribution of runtimes for the successful executions of the implemented algorithms



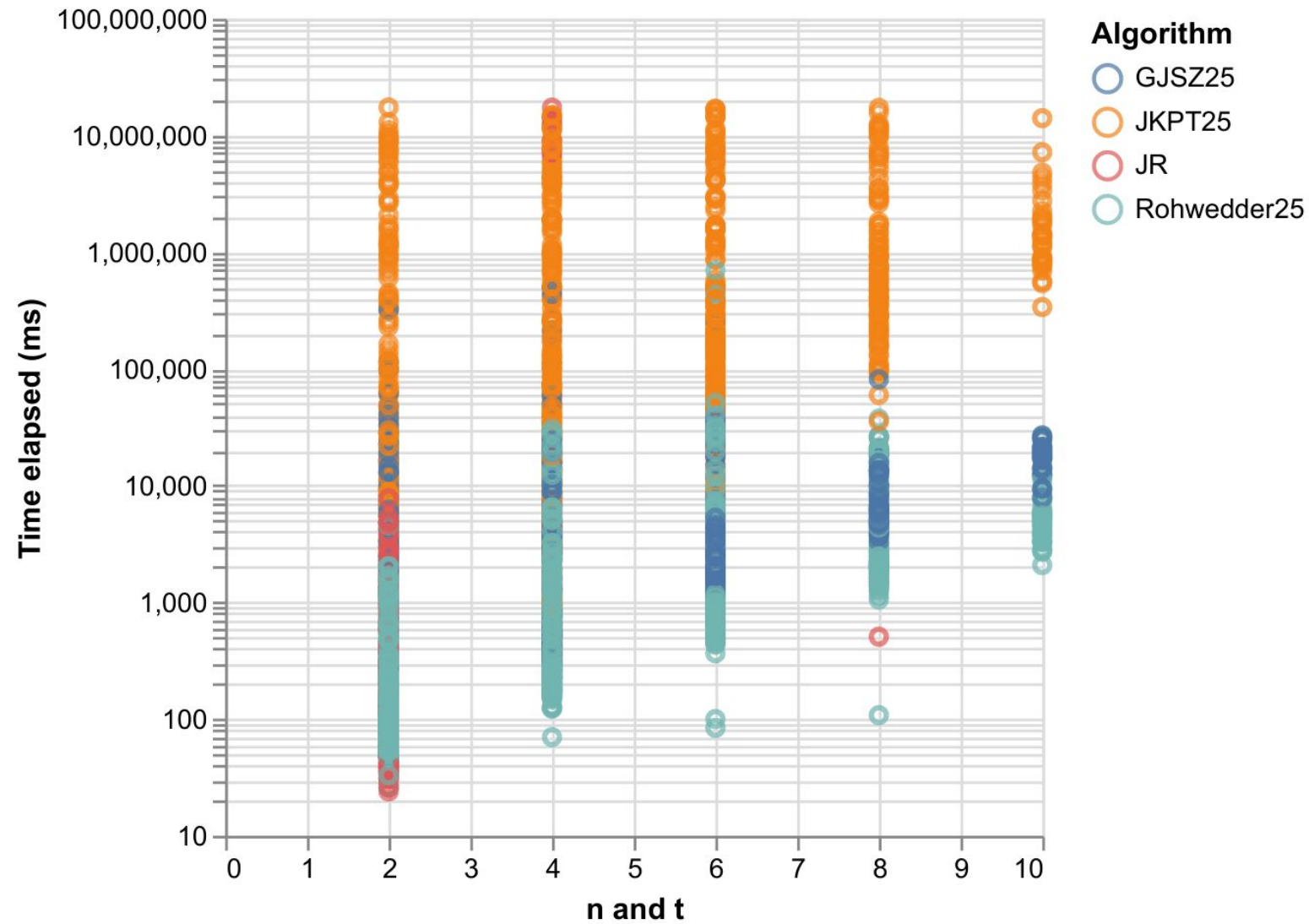
Comparisons of input factors: Δ_A, Δ_b

- Rohwedder25: low runtimes across the full tested range
- JR & GJSZ25: success rates decrease as Δ_A, Δ_b increase
- JR: almost no successful executions above $\Delta_A, \Delta_b > 100$
- JKPT25: very high runtimes; limited effect of increasing Δ_b , consistent with logarithmic dependence

Runtime comparison for different values of Δ_A and Δ_b

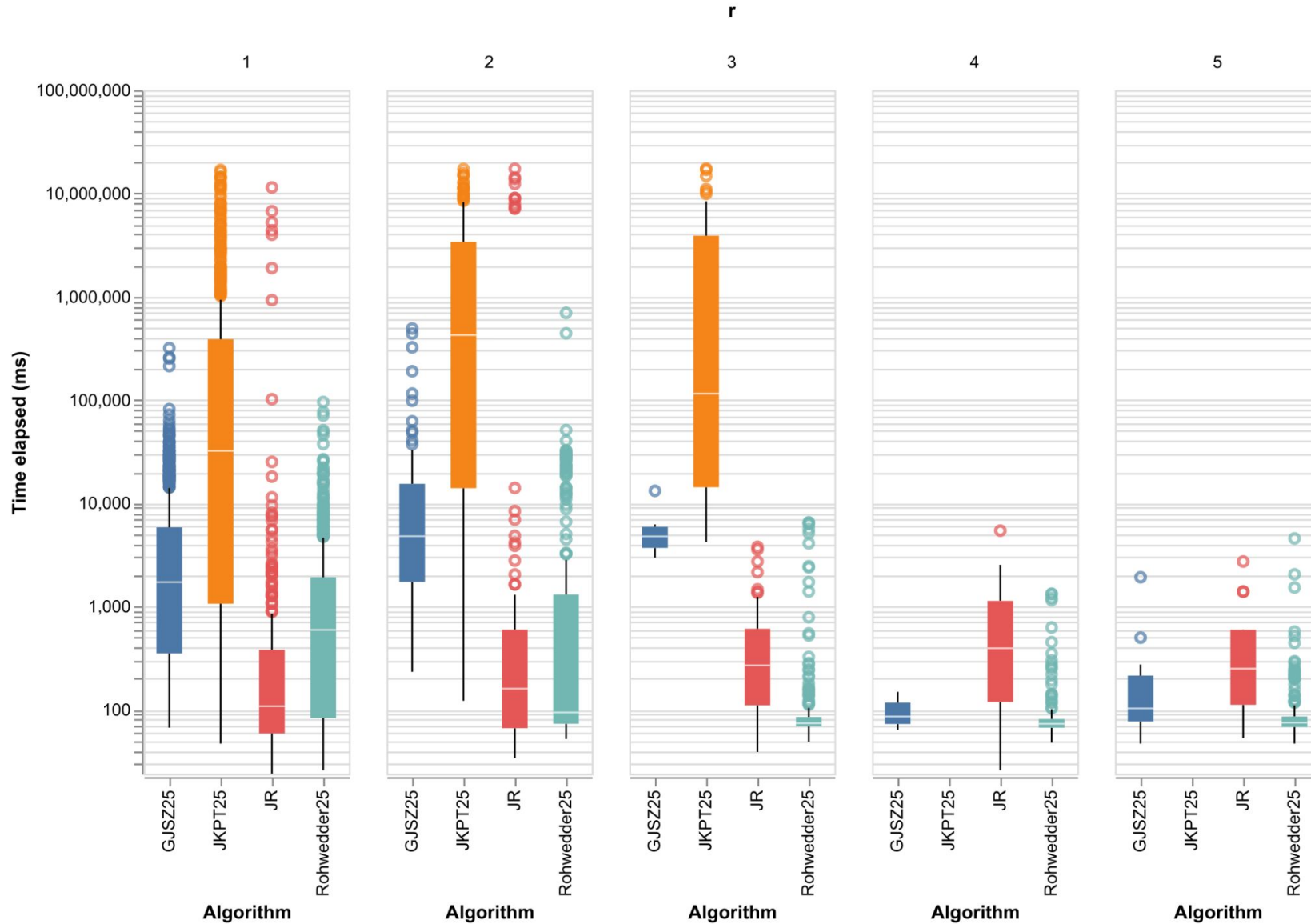
Comparison of input factors: n and t

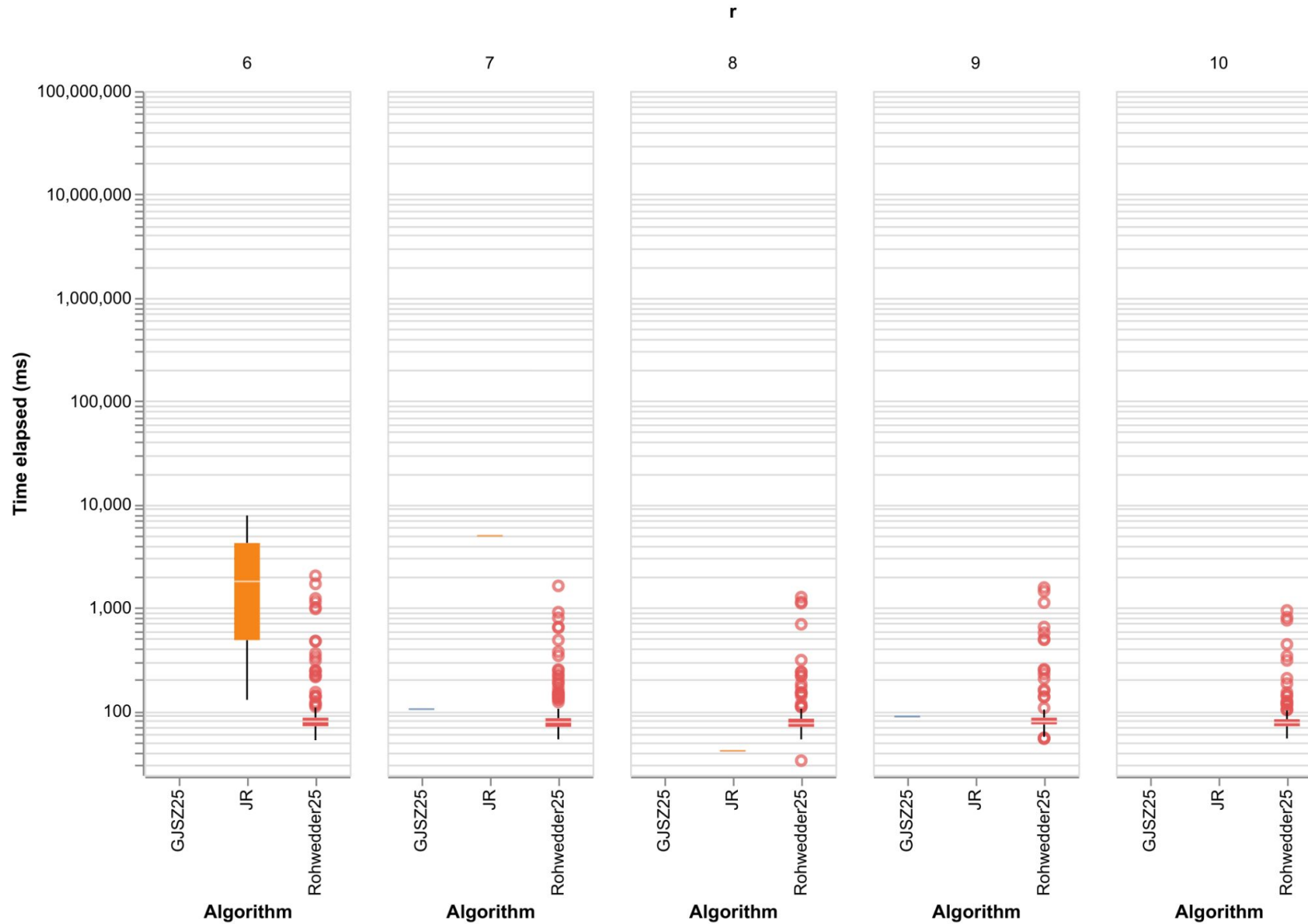
- n and t increase simultaneously, determining matrix width $m = n \cdot t = n^2$
- Runtime generally increases with increasing matrix width
- JR: few successful executions beyond $n = 2$
- Runtime increases linearly



Comparison of input factors: r

- r scales the algorithms complexity **exponentially**
- Compare results per value of r up to 10
 - JKPT25: almost no successful executions for $r > 3$
 - JR: very few successful executions beyond $r = 6$
 - GJSZ25: very few successful executions beyond $r = 5$
 - Rohwedder25: successful executions across all tested r values





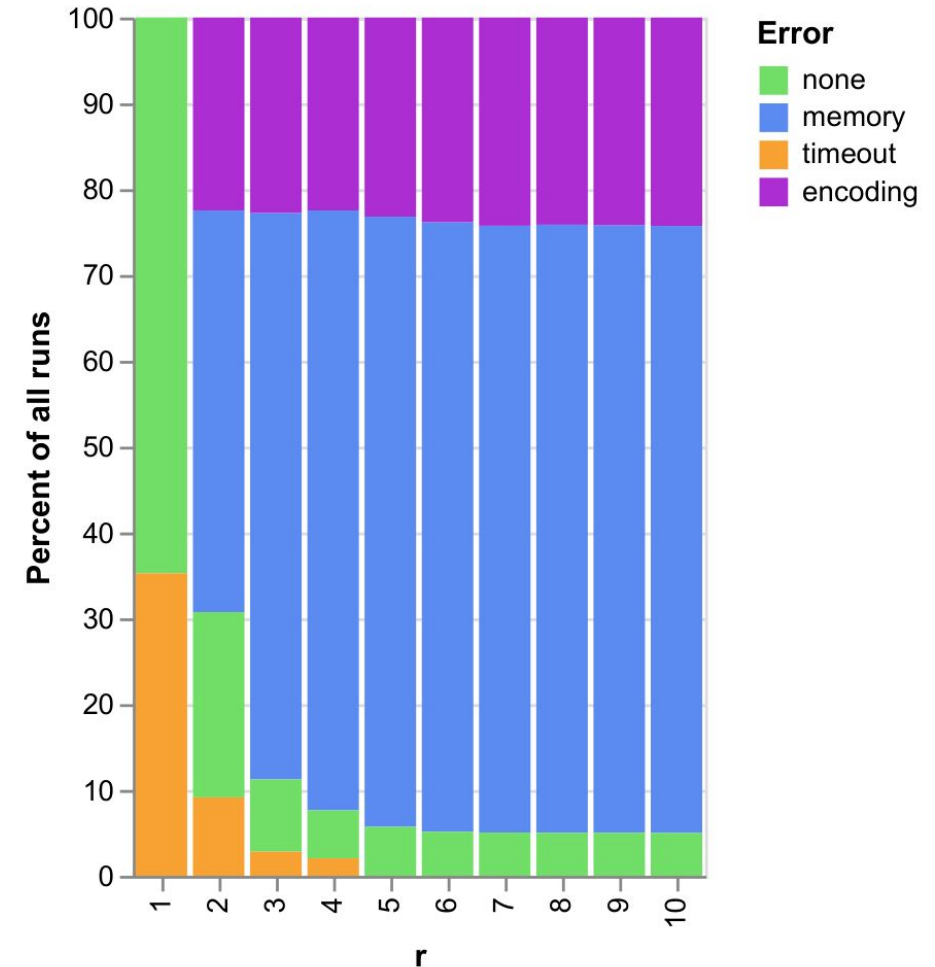
Comparison of input factors: r (Cont.)

- Rohwedder25 solves a lot of instances too demanding for other algorithms
 - 20% success rate constant for values of r above 4
- GJSZ25 has a very high success rate for $r = 1$
- JR pulls ahead of JKPT25 at $r = 3$

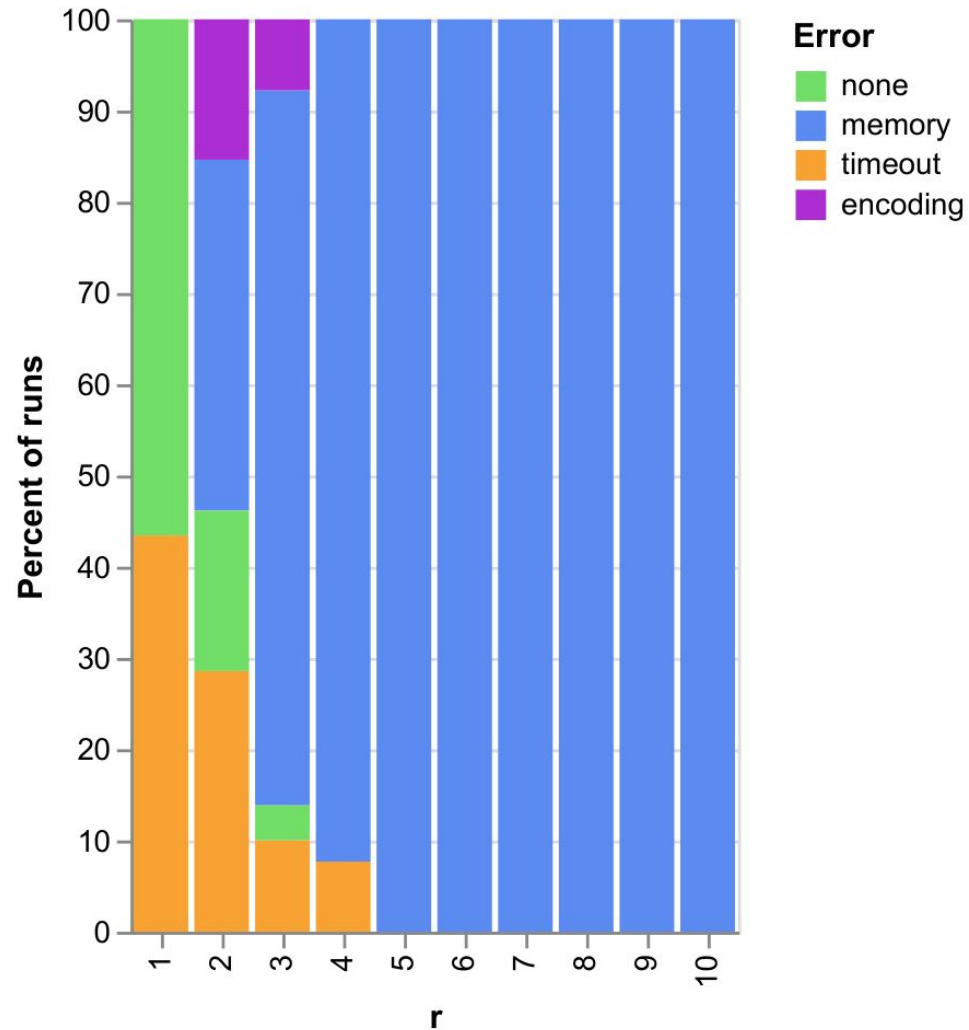
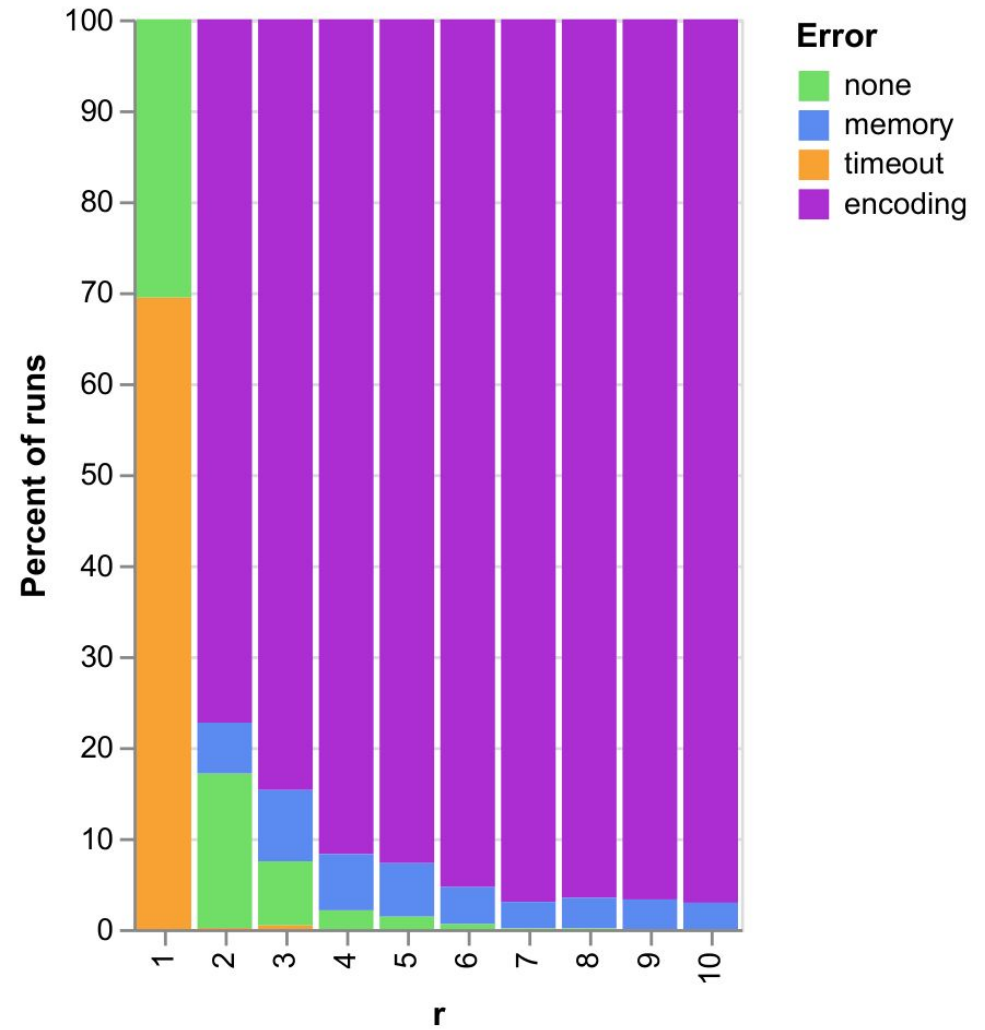
Algorithm	r	Median (ms)	Mean (ms)	Success (%)
JKPT25	1	32 134.00	767 958.86	56.56
	2	424 870.50	2 322 605.84	17.56
	3	115 323.50	2 799 930.02	3.80
JR	1	108.00	131 678.19	30.57
	2	160.00	855 096.63	16.98
	3	269.00	536.46	7.04
	≥ 4	394.00	1 846.23	0.83
GJSZ25	1	1 719.00	7 004.67	82.62
	2	4 789.00	22 144.57	15.61
	3	4 769.00	5 361.55	1.11
	≥ 4	95.50	135.24	0.63
Rohwedder25	1	592.00	2 396.89	91.67
	2	94.00	7 050.64	37.82
	3	74.00	278.54	22.22
	≥ 4	77.00	110.52	20.00

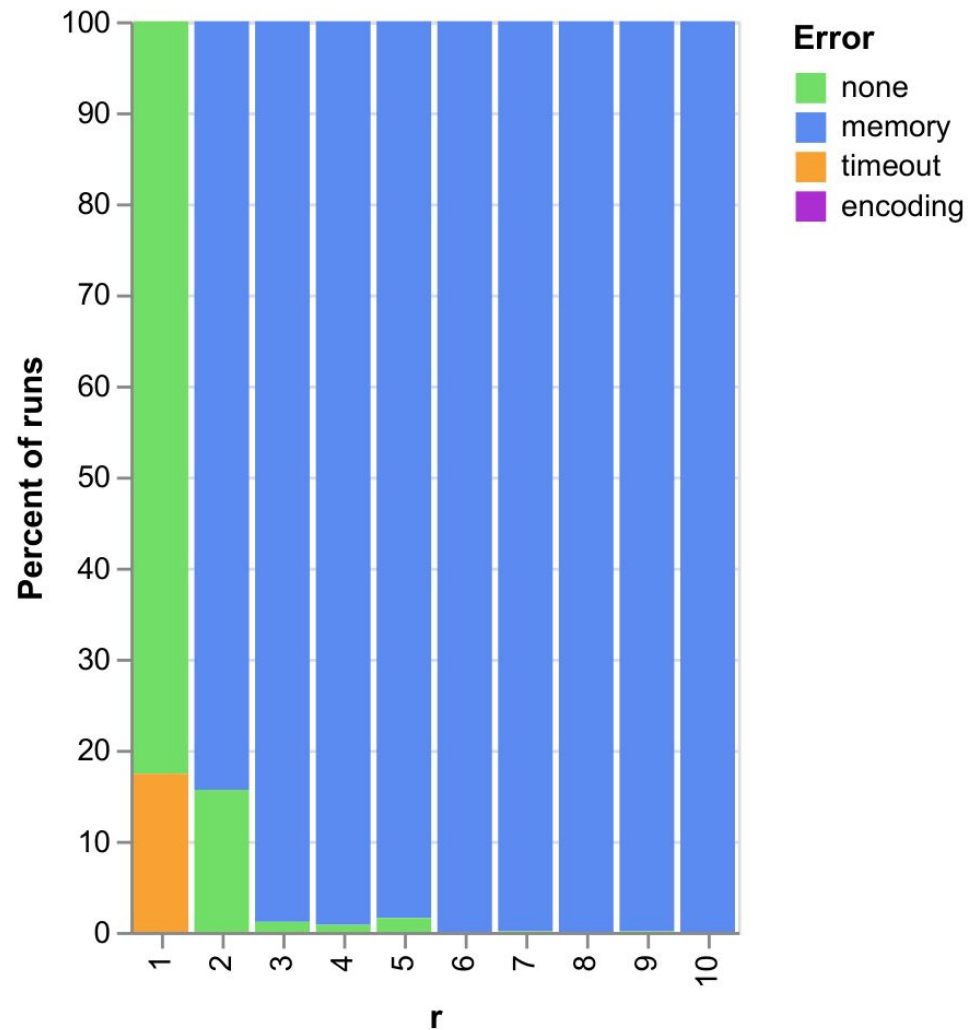
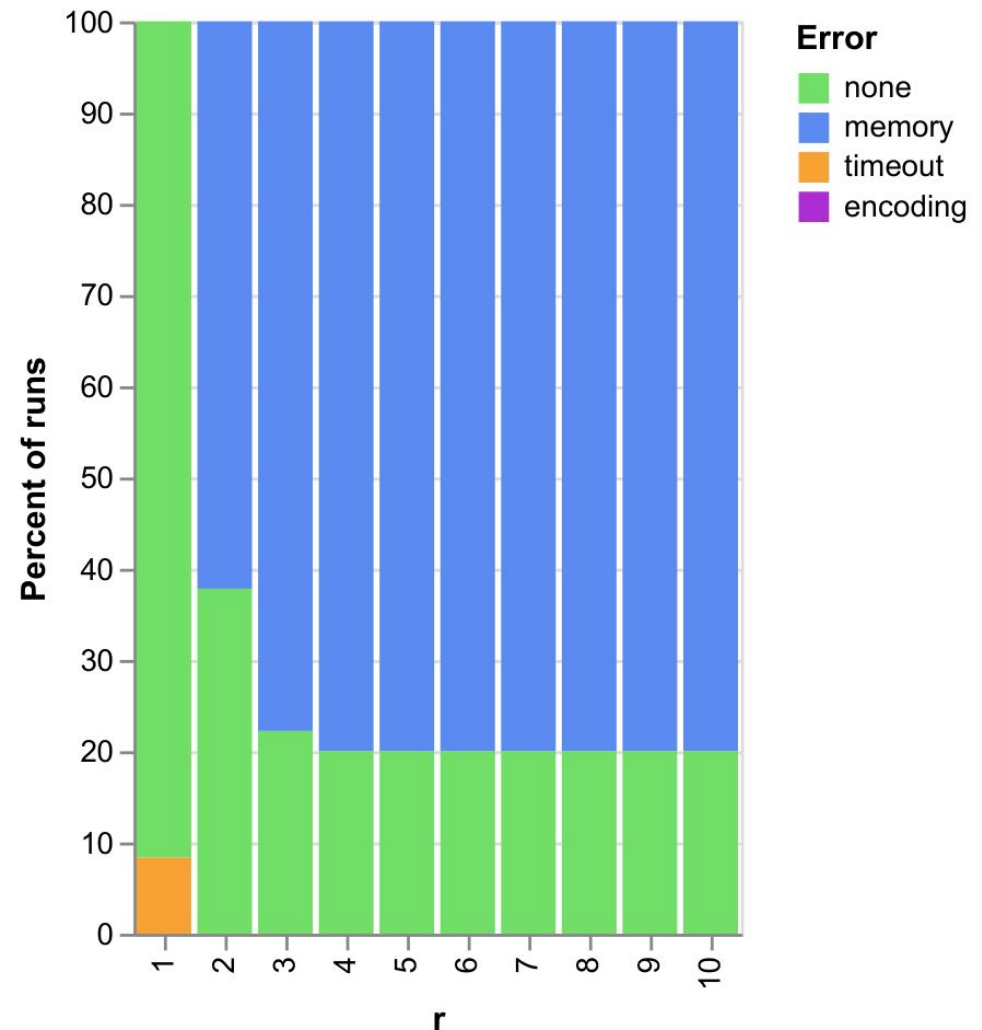
Evaluation of errors

- Four different kinds of errors:
 - None: The algorithm completed successfully and produced a valid result.
 - Memory error: The execution failed because the algorithm exceeded the available memory.
 - Timeout: The execution did not finish within the specified time limit.
 - Encoding: The execution failed due to an error related to the chosen encoding method
- Errors were checked via the .out files from the cluster batch processing



Distribution of errors for different values of r

Distribution of errors for different values of r (JKPT25)Distribution of errors for different values of r (JR)

Distribution of errors for different values of r (GJSZ25)Distribution of errors for different values of r (Rohwedder25)

Conclusion

- **JKPT25:** generally high runtimes, but comparatively little impact from increasing Δ_b
- **JR:** very fast when successful, but limited success rate due in part to the chosen encoding
- **GJSZ25:** high success rate for small r , but success rate drops sharply as r increases
- **Rohwedder25:** low runtimes, highest success rate, and fewest memory-related failures
- BitSet encoding saves memory but limits the number of representable states
- Overall: memory limitations are the dominant source of failures; alternative state/graph representations could reduce these requirements

Thank you for listening!

Any questions?

References

- [1] S. Islam, S. Balasubramaniam, S. Gupta, S. Brajesh, R. Badlani, N. Labhishetty, A. Baid, P. Goyal, and N. Goyal. Pattern-based automatic parallelization of representative-based clustering algorithms. In 2018 IEEE 5th International Conference on Data Science and Advanced Analytics (DSAA), pages 99–108, 2018.
- [2] N. Kataev. Interactive parallelization of c programs in SAPFOR. In SSI, pages 139–148, 2020.
- [3] O. Oey, M. Huebner, T. Stripf, and J. Becker. Embedded Multi-Core Code Generation with Cross-Layer Parallelization. In 15th Workshop on Parallel Programming and Run-Time Management Techniques for Many-Core Architectures and 13th Workshop on Design Tools and Architectures for Multicore Embedded Computing Platforms (PARMA-DITAM 2024), volume 116 of Open Access Series in Informatics (OASIs), pages 5:1–5:13, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [4] Y. Xu, H. Lee, D. Chen, B. A. Hechtman, Y. Huang, R. Joshi, M. Krikun, D. Lepikhin, A. Ly, M. Maggioni, R. Pang, N. Shazeer, S. Wang, T. Wang, Y. Wu, and Z. Chen. GSPMD: general and scalable parallelization for ML computation graphs. CoRR, abs/2105.04663, 2021.

Appendix

- **Resources per task:**
 - 1 CPU core
 - 2 GB memory
 - 5-hour wall-clock limit
 - Maximum 100 concurrent tasks